LEZIONE 17
Unit and Integration Testing: Frameworks

Ingegneria del Software 2 – Software Testing

Università degli Studi di Tor Vergata - Roma

Guglielmo De Angelis
guglielmo.deangelis@iasi.cnr.it

test di integrazione

- obiettivo
  - far emergere malfunzionamenti derivanti dalla interazione di due o più moduli/componenti del SUT
  - acquisire confidenza che:
    - sussistano le dovute interconnessioni tra moduli/componenti del SUT
    - le interazioni tra i moduli/componenti del SUT rispettino i “contratti funzionali” stabiliti dai protocolli/specifiche di interazione
- assunzione
  - ogni modulo/unità funzionale del SUT in “isolamento” si comporti correttamente (test di unità passati)
- chi, come, quando
  - sviluppatori, integratori, tester … dipende dagli scenari di integrazione considerati
  - in generale i test possono essere redatti assumendo la disponibilità del codice di tutti i moduli considerati (white-box) o solo le specifiche di interazione (black-box)
    - nella pratica spesso si devono tener presente entrambe le possibilità contemporaneamente
  - necessità di **stub/mock** e di framework per la loro gestione
    - il test è fatto modo incrementale considerando solo parti del SUT
    - le interazioni potrebbero richiedere moduli/componenti non disponibili o che non si vogliono considerare
    - uno stub è un elemento software definito in fase di test (e.g. classe, modulo, unità, etc.)
      - non è soggetto a test ma fa parte dell’enviromnet di test
      - mima il comportamento dell’unità mancante … spesso attraverso comportamenti banali o costanti
  - **test driver** avanzati
    - configurazione dell’ambiente di test
    - coordinamento delle risorse
    - clean-up dopo l’esecuzione
    - in un test driver è l’esecutore di un test, spesso codificata parte della logica di integrazione che si vuole ottenere dai moduli/composti

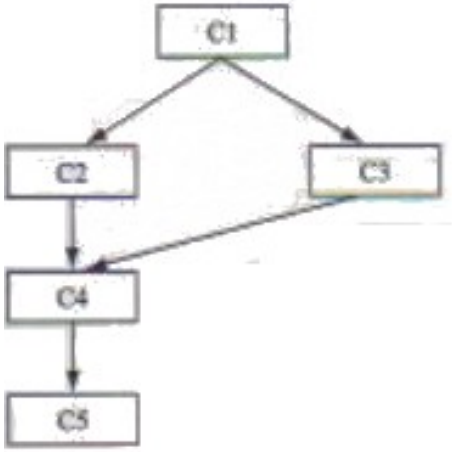
strategie per integration testing

- identificare la soluzione ottima è in generale è un problema difficile
  - NP-complete
- strategie comuni

a) big bang: integrare tutte le componenti in un passo e testare il funzionamento

b) top-down:
- integrazione delle interfacce
- test attraverso funzionalità più generali o più vicine ai confine del SUT
- le sequenze di test sono definite a partire dagli scenari d'uso del SUT
- il raffinamento avviene sostituendo implementazioni più dettagliate delle interfacce esposte o ampliando internamente l'insieme dei moduli considerati
- l'uso di **stub** simula i comportamenti/funzionalità mancanti

c) bottom-up:
- integrazione di unità elementari
- test di integrazioni più lontane dai bordi del sistema
- l'uso di **test driver** simula il comportamento di moduli/componenti più esterni del sistema
- l'eventuale riferimento a comportamenti/funzionalità mancanti è emulato per mezzo di **stub**
- il raffinamento avviene considerando interfacce più generali o focalizzandosi sulle parti più periferiche del sistema



(a)

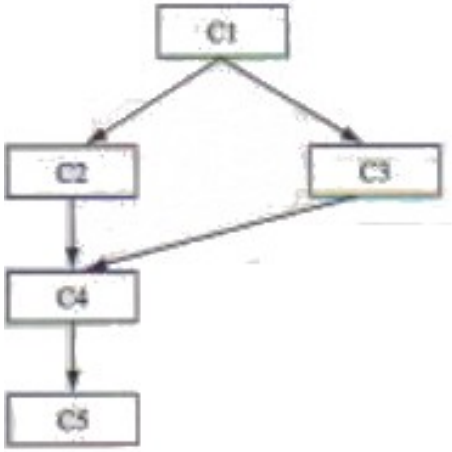
strategie per integration testing

- identificare la soluzione ottima è in generale è un problema difficile
  - NP-complete
- strategie comuni

a) big bang: integrare tutte le componenti in un passo e testare il funzionamento

b) top-down:
- integrazione delle interfacce
- test attraverso funzionalità più generali o più vicine ai confine del SUT
- le sequenze di test sono definite a partire dagli scenari d'uso del SUT
- il raffinamento avviene sostituendo implementazioni più dettagliate delle interfacce esposte o ampliando internamente l'insieme dei moduli considerati
- l'uso di **stub** simula i comportamenti/funzionalità mancanti

c) bottom-up:
- integrazione di unità elementari
- test di integrazioni più lontane dai bordi del sistema
- l'uso di **test driver** simula il comportamento di moduli/componenti più esterni del sistema
- l'eventuale riferimento a comportamenti/funzionalità mancanti è emulato per mezzo di **stub**
- il raffinamento avviene considerando interfacce più generali o focalizzandosi sulle parti più periferiche del sistema



(a)

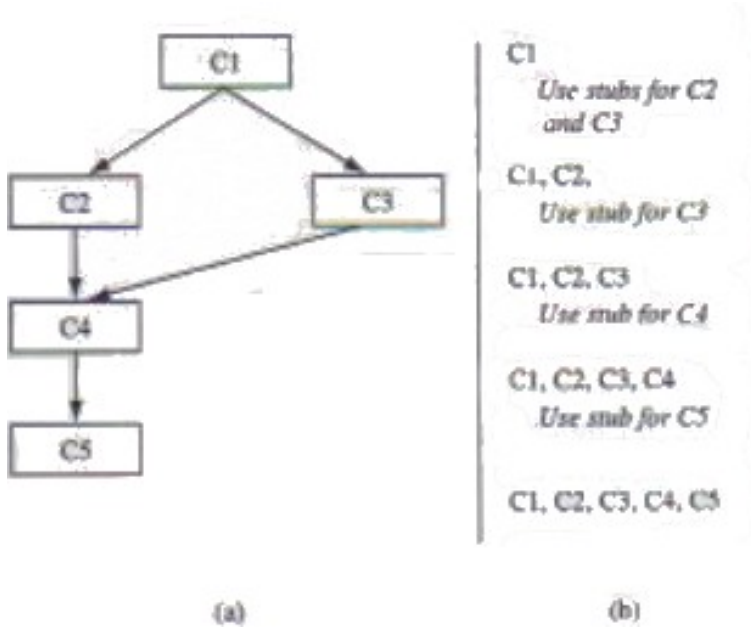
strategie per integration testing

- identificare la soluzione ottima è in generale è un problema difficile
  - NP-complete
- strategie comuni

a) big bang: integrare tutte le componenti in un passo e testare il funzionamento

b) top-down:
- integrazione delle interfacce
- test attraverso funzionalità più generali o più vicine ai confine del SUT
- le sequenze di test sono definite a partire dagli scenari d'uso del SUT
- il raffinamento avviene sostituendo implementazioni più dettagliate delle interfacce esposte o ampliando internamente l'insieme dei moduli considerati
- l'uso di **stub** simula i comportamenti/funzionalità mancanti

c) bottom-up:
- integrazione di unità elementari
- test di integrazioni più lontane dai bordi del sistema
- l'uso di **test driver** simula il comportamento di moduli/componenti più esterni del sistema
- l'eventuale riferimento a comportamenti/funzionalità mancanti è emulato per mezzo di **stub**
- il raffinamento avviene considerando interfacce più generali o focalizzandosi sulle parti più periferiche del sistema



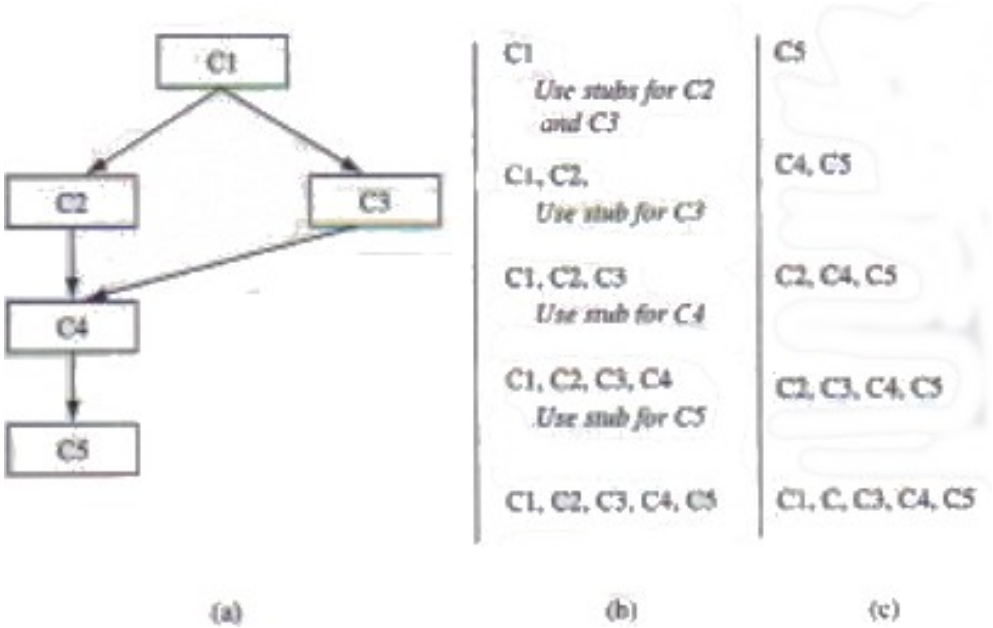
strategie per integration testing

- identificare la soluzione ottima è in generale è un problema difficile
  - NP-complete
- strategie comuni

a) big bang: integrare tutte le componenti in un passo e testare il funzionamento

b) top-down:
- integrazione delle interfacce
- test attraverso funzionalità più generali o più vicine ai confine del SUT
- le sequenze di test sono definite a partire dagli scenari d'uso del SUT
- il raffinamento avviene sostituendo implementazioni più dettagliate delle interfacce esposte o ampliando internamente l'insieme dei moduli considerati
- l'uso di **stub** simula i comportamenti/funzionalità mancanti

c) bottom-up:
- integrazione di unità elementari
- test di integrazioni più lontane dai bordi del sistema
- l'uso di **test driver** simula il comportamento di moduli/componenti più esterni del sistema
- l'eventuale riferimento a comportamenti/funzionalità mancanti è emulato per mezzo di **stub**
- il raffinamento avviene considerando interfacce più generali o focalizzandosi sulle parti più periferiche del sistema



test di integrazione in O.O. – strumenti

test di integrazione in O.O. – strumenti

- test driver



- stubbing/mocking



– https://site.mockito.org/

mockito – 1

framework per la definizione di stub a supporto dello sviluppo di test in Java

- fornisce un insieme di primitive che consentono ai tester di
  - dichiarare stub/mock
  - definire valori di ritorno per operazioni su specifiche istanze
  - (ri)definire comportamenti per operazioni in base a specifiche istanze in input
- supporto nativo a paradigmi di programmazione agile guidati dalla scrittura di casi di test
    - Test Driven Development (TDD)
    - Behavior Driven Development (BDD)

mockito – 2

costrutti chiave del framework sono:

- creazione programmatica dei mock/stub
  - `mock()`
  - `spy()`
- dichiarazioni semplificate per mezzo di annotazioni sintattiche
  - `@Mock`
  - `@Spy`
  - `@Captor`
  - `@InjectMocks`
- definizione programmatica dei valori di ritorno
  - `Mockito.when(mockedInstance.fooOperation()).thenReturn(true)`
  - `BDDMockito.given(mockedInstance.fooOperation()).willReturn(true)`
- programmazione personalizzata di comportamenti
  - `Mockito.when(mockedInstance.fooOperation()).thenAnswer(new Answer<Object>() { … })`
- verifica progammatica se i mock sono stati usati/riferiti
  - `Mockito.verify(mockedInstance).fooOperation()`
  - `BDDMockito.then(mockedInstance).should().fooOperation()`
- integrazione nativa con Junit
  - `MockitoJUnitRunner`
  - `MockitoJUnit.rule()`
- … per tutto il resto c'è una notevole quantità di documentazione online … ufficiale e non!!

mockito – 2

costrutti chiave del framework sono:

- creazione programmatica dei mock/stub
  - **`mock()`**
  - **`spy()`**
- dichiarazioni semplificate
  - **`@Mock`**
  - **`@Spy`**
  - **`@Captor`**
  - **`@InjectMocks`**
- definizione programmatica del comportamento
  - **`Mockito.when(m`**
  - **`BDDMockito.giv`**
- programmazione personalizzata di comportamenti
  - **`Mockito.when`**`(mockedInstance.fooOperation()).`**`thenAnswer`**`(new` **`Answer`**`<Object>() { … })`
- verifica progammatica se i mock sono stati usati/riferiti
  - **`Mockito.verify`**`(mockedInstance).fooOperation()`
  - **`BDDMockito.then`**`(mockedInstance).`**`should`**`().fooOperation()`
- integrazione nativa con Junit
  - **`MockitoJUnitRunner`**
  - **`MockitoJUnit.rule`**`()`
- … per tutto il resto c'è una notevole quantità di documentazione online … ufficiale e non!!

```
//Let's import Mockito statically so that the code looks clearer
import static org.mockito.Mockito.*;

//mock creation
List mockedList = mock(List.class);


//You can mock concrete classes, not just interfaces
LinkedList mockedLinkedList = mock(LinkedList.class);
```

mockito – 2

costrutti chiave del framework sono:

- creazione programmatica dei mock/stub
  - `mock()`
  - `spy()`
- dichiarazioni semplificate per mezzo di annotazioni sintattiche
  - `@Mock`
  - `@Spy`
  - `@Captor`
  - `@InjectMocks`
- definizione programmatica dei valori di ritorno
  - `Mockito.when(mockedInstance.fooOperation()).thenReturn(true)`
  - `BDDMockito.given(mockedInstance.fooOperation()).willReturn(true)`
- programmazione personalizzata di comportamenti
  - `Mockito.when(mockedInstance.fooOperation()).thenAnswer(new Answer<Object>() { … })`
- verifica progammatica se i mock sono stati usati/riferiti
  - `Mockito.verify(mockedInstance).fooOperation()`
  - `BDDMockito.then(mockedInstance).should().fooOperation()`
- integrazione nativa con Junit
  - `MockitoJUnitRunner`
  - `MockitoJUnit.rule()`
- … per tutto il resto c'è una notevole quantità di documentazione online … ufficiale e non!!

mockito – 2

costrutti chiave del framework sono:

- creazione programmatica dei mock/stub
  - `mock()`
  - `spy()`
- dichiarazioni semplificate per mezzo di annotazioni sintattiche
  - `@Mock`
  - `@Spy`
  - `@Captor`
  - `@InjectMocks`
- definizione programmatica dei valori di ritorno
  - `Mockito.when(mockedInstance.fooOperation()).thenReturn(true)`
  - `BDDMockito.given(mockedInstance.fooOperation()).willReturn(true)`
- programmazione personalizzata di comportamenti
  - `Mockito.when(mockedInstance.fooOperation()).thenAnswer(new Answer<Object>() { … })`
- verifica progammatica se i mock sono stati usati/riferiti
  - `Mockito.verify(mockedInstance).fooOperation()`
  - `BDDMockito.then(mockedInstance).should().fooOperation()`
- integrazione nativa con Junit
  - `MockitoJUnitRunner`
  - `MockitoJUnit.rule()`
- … per tutto il resto c'è una notevole quantità di documentazione online … ufficiale e non!!

mockito – 2

costrutti chiave del framework sono:

- creazione programmatica dei mock/stub
  - `mock()`
  - `spy()`
- dichiarazioni semplificate per mezzo di annotazioni sintattiche
  - `@Mock`
  - `@Spy`
  - `@Captor`
  - `@InjectMocks`
- definizione programmatica dei valori di ritorno
  - `Mockito.when(mockedInstance.fooOperation()).thenReturn(true)`
  - `BDDMockito.given(mockedInstance.fooOperation()).willReturn(true)`
- programmazione personalizzata di comportamenti
  - `Mockito.when(mockedInstance.fooOperation()).thenAnswer(new Answer<Object>() { … })`
- verifica progammatica se i mock sono stati usati/riferiti
  - `Mockito.verify(mockedInstance).fooOperation()`
  - `BDDMockito.then(mockedInstance).should().fooOperation()`
- integrazione nativa con Junit
  - `MockitoJUnitRunner`
  - `MockitoJUnit.rule()`
- … per tutto il resto c'è una notevole quantità di documentazione online … ufficiale e non!!

mockito – 2

costrutti chiave del framework sono:

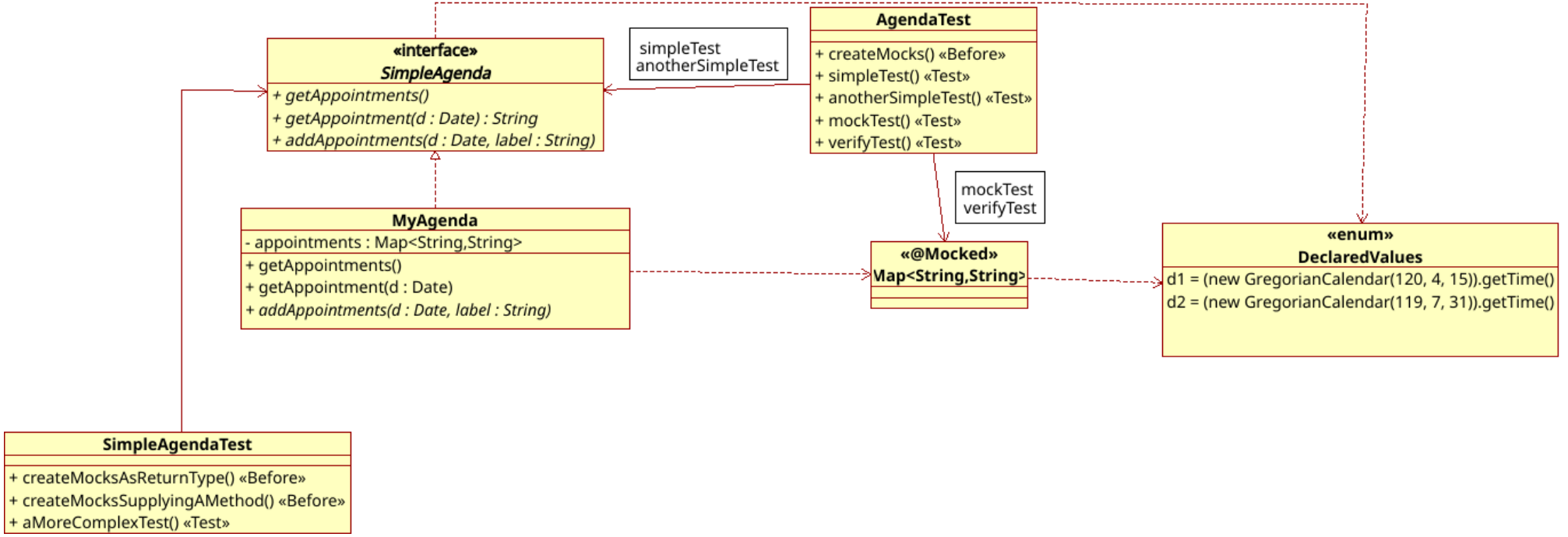
- creazione programmatica dei mock/stub
  - `mock()`
  - `spy()`
- dichiarazioni semplificate per mezzo di annotazioni sintattiche
  - `@Mock`
  - `@Spy`
  - `@Captor`
  - `@InjectMocks`
- definizione programmatica dei valori di ritorno
  - `Mockito.when(mockedInstance.fooOperation()).thenReturn(true)`
  - `BDDMockito.given(mockedInstance.fooOperation()).willReturn(true)`
- programmazione personalizzata di comportamenti
  - `Mockito.when(mockedInstance.fooOperation()).thenAnswer(new Answer<Object>() { … })`
- verifica progammatica se i mock sono stati usati/riferiti
  - `Mockito.verify(mockedInstance).fooOperation()`
  - `BDDMockito.then(mockedInstance).should().fooOperation()`
- integrazione nativa con Junit
  - `MockitoJUnitRunner`
  - `MockitoJUnit.rule()`
- … per tutto il resto c'è una notevole quantità di documentazione online … ufficiale e non!!

esempi mockito

VEDERE MATERIALE ALLEGATO ALLA LEZIONE :

it.uniroma2.dicii.is2.tests.agenda.SimpleAgendaTest.java

it.uniroma2.dicii.is2.tests.agenda.AgendaTest.java



automate the build – framework

- MAVEN (da Lezione 7 e Lezione 10)
  - sistema di build
  - gestione delle dipendenze attraverso istanze di service registry
    - MAVEN Central
  - un progetto/modulo MAVEN è specificato mediante un descrittore:
    - `pom.xml`
  - framework modulare strutturato in plug-in
    - ogni plug-in aumenta le funzionalità del framework rispetto ad un determinato aspetto
    - i plug-in si innestano in una o più fasi del processo di build previsto da MAVEN



automate the build – framework

MAVEN fasi (da Lezione 7)

- il processo di build è strutturato in fasi
  - le fasi sono concettualmente organizzate in sequenza
  - tipicamente errori nell'esecuzione di una fase comportano il fallimento dell'intero processo di build
- ogni step del processo di build esplicitamente indica la fase di appartenenza
- fasi di default
  - validate: validate the project is correct and all necessary information is available
  - compile: compile the source code of the project
  - test: test the compiled source code using a suitable unit testing framework.
  - package: take the compiled code and package it in its distributable format, such as a JAR.
  - integration-test: process and deploy the package if necessary into an environment where integration tests can be run
  - verify: run any checks to verify the package is valid and meets quality criteria
  - install: install the package into the local repository, for use as a dependency in other projects locally
  - deploy: done in an integration or release environment, copies the final package to the remote repository
  - clean: cleans up artifacts created by prior build
  - site: generates site documentation for this project
- è possibile dichiarare ulteriori fasi
  - realisticamente nel nostro corso sarà sufficiente considerarne solo alcune
- ulteriori dettagli su:
  - https://maven.apache.org/guides/getting-started/maven-in-five-minutes.html
  - https://maven.apache.org/guides/getting-started/index.html



unit e integration testing con MVN

- Surefire Plugin
  - https://maven.apache.org/surefire/maven-surefire-plugin/
- concepito principalmente per eseguire test di unità
- in pratica è utile per lanciare test durante il build di un unico progetto/modulo Maven

- Failsafe Plugin
  - https://maven.apache.org/surefire/maven-failsafe-plugin/
- concepito per eseguire test di integrazione
- in pratica è utile per lanciare test che:
  - a seguito del build di più progetti/moduli Maven
  - configurare l'ambiente di test attraverso fasi specifiche:
    - set-up dell'ambiente per l'integration test
      - avvio di application container (e.g. jetty, tomcat, etc)
      - deploy e launch di servizi o altri moduli
    - azioni di clean-up e teardown per rimuovere effetti causati dalla configurazione ed esecuzione dei test di integrazione

Maven Surefire Plugin – 1

- viene attivato durante la fase `test` del ciclo di build Maven
  - vedi fasi di build Maven e loro ordinamento in Lezione 7
- scansiona il progetto Maven cercando i test JUnit da lanciare
  - tipicamente collocati nella directory:
    - `${basedir}/src/test/`
  - metodi pubblici annotati con `@Test`
  - per default i nomi delle classi con metodi di test nel formato:
    - `Test*`
    - `*Test`
    - `*Tests`
    - `*TestCase`
- i report generati durante l'esecuzione sono (generarlmente) generati in
  - `${basedir}/target/surefire-reports/TEST-*.xml`

Maven Surefire Plugin – 2

```
1. <project>
2.   [...]
3.   <build>
4.     <pluginManagement>
5.       <plugins>
6.         <plugin>
7.           <groupId>org.apache.maven.plugins</groupId>
8.           <artifactId>maven-surefire-plugin</artifactId>
9.           <version>3.0.0-M4</version>
10.        </plugin>
11.      </plugins>
12.    </pluginManagement>
13.  </build>
14.  [...]
15. </project>
```

```
1. <dependencies>
2.   [...]
3.     <dependency>
4.       <groupId>junit</groupId>
5.       <artifactId>junit</artifactId>
6.       <version>4.11</version>
7.       <scope>test</scope>
8.     </dependency>
9.   [...]
10. </dependencies>
```

Maven Failsafe Plugin – 1

- viene attivato durante la fase `integration-test/verify` del ciclo di build Maven
  - vedi fasi di build Maven e loro ordinamento in Lezione 7
- scansiona il progetto Maven cercando i test JUnit da lanciare
  - tipicamente collocati nella directory:
    - `${basedir}/src/test/`
  - metodi pubblici annotati con `@Test`
  - per default i nomi delle classi con metodi di test nel formato:
    - `IT*`
    - `*IT`
    - `*ITCase`
- i report generati durante l'esecuzione sono (generarlmente) generati in
  - `${basedir}/target/failsafe-reports/failsafe-summary.xml`

Maven Failsafe Plugin – 2

```
<project>
  [...]
  <build>
    <plugins>
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-failsafe-plugin</artifactId>
        <version>3.0.0-M4</version>
        <executions>
          <execution>
            <goals>
              <goal>integration-test</goal>
              <goal>verify</goal>
            </goals>
          </execution>
        </executions>
      </plugin>
    </plugins>
  </build>
  [...]
</project>
```

```
<dependencies>
  [...]
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.11</version>
    </dependency>
  [...]
</dependencies>
```

esplicite raccomandazioni

nello svolgimento del progetto è fortemente consigliato sperimentare:

- la progettazione e la realizzazione di test di unità associandoli al processo di CI attraverso Maven Surefire
- la progettazione e la realizzazione di test di integrazione associandoli al processo di CI attraverso Maven Failsafe
- l’utilizzo di mock per test di unità ed integrazione sfruttando le principali funzionalità offerte da Mockito